ParkOps D1: Design

EECS 3311 B, Fall 2026 · Deliverable 1: design and executable proposal path · Route A (ParkOps) · Bryan Maristanez · October 2026

1. Scope

D1 implements the validated domain model and the executable proposal path for the supplied fixture. `StudentApplication.propose(String)` reads booking B1, evaluates a target space under policy version 1, and stores a `PENDING` proposal that records the booking and policy versions it observed. The booking never moves; approval and execution are D2 work. `bash run.sh run scripted` demonstrates the accepted target B12 and the rejected targets D09 and Z99 using replayed input, which is not a live model run. Figure 1 (class diagram) and Figure 2 (sequence diagram) follow the text on separate pages.

2. Responsibilities

Class	Responsibility	How its state is protected
`DomainRules` (starter TODOs completed)	The single home for the identifier rule `[A-Z][A-Z0-9_-]{0,15}` and the 1–24 hour duration rule.	Private constructor; static guards throw `IllegalArgumentException`; the pattern is compiled once as a constant.
`Booking`	One booking: id, assigned space, duration, version and accessibility requirement.	The constructor validates every argument before assigning; all fields are `private final`; getters only.
`Space`	One space's id and open, occupied and accessible flags. `isAvailable()` is the only definition of “available”: open and not occupied.	Validated id; `private final` fields; no setters.
`ProposalStatus`	The four proposal states: `PENDING`, `APPROVED`, `REJECTED`, `EXECUTED`.	An enum, so no other status value can exist.
`Proposal`	A proposed reassignment, kept separate from the `Booking` it would change. Records its id, the booking, the target space, the observed booking and policy versions, and its status.	Ids and versions are validated; the constructor always sets `PENDING` and there is no setter; `toView()` hands out an immutable `ProposalView`.
`EligibilityPolicy`	Policy version 1: a target is eligible when it is not the booking's current space and is available. Carries its own version number.	Stateless; `isEligible` changes nothing.
`StudentApplication` (starter class rewritten)	Owns the in-memory state of one run: builds a `Booking` and four `Space` objects from `Fixture`, stores proposals, generates ids `P1`, `P2`, …, and implements `propose`.	Collections are private. Callers receive read-only copies from `bookingSnapshot()`, `spaceSnapshot(String)` and `proposals()`. Each instance builds its own objects, so two applications never share state.
`Main` (starter, `scripted` mode extended)	Demonstration: feeds three replayed requests (B12, D09, Z99) to `propose` and prints the result and the booking after each.	Never approves or executes.

3. Design decisions and the alternatives rejected

Alternative considered: where the eligibility rule lives

The rule could have been an `if` inside `StudentApplication.propose`, or a method such as `Space.isEligibleFor(Booking)`. It was placed in its own class, `EligibilityPolicy`, with its version number beside it. D3 replaces policy version 1 with version 2, which also requires an accessible target when the booking requires one, and must reject proposals created under the old version. With the rule and its version in one object, `propose` records `policy.getVersion()` in every proposal, D3 changes a single class, and D2's list-alternatives tool can reuse `isEligible`. An inline `if` would have mixed the rule with id generation and storage, and a method on `Space` would have made a space depend on a booking policy it should not know about.

Decision	Alternative rejected	Reason
The status is the enum `ProposalStatus`	A `String` status	A misspelt status cannot compile, and no status outside the brief's four can exist. `toView()` converts with `status.name()` because the supplied `ProposalView` stores a `String`.
`Proposal`'s constructor always sets `PENDING`; no setter	Status as a constructor parameter, or a `setStatus` method	No caller can create an already-approved proposal; the brief says approval never comes from the model. D2 will add controlled transitions that are legal only from `PENDING`.
Callers receive copies: snapshot maps and a list of `ProposalView` records	Returning the `Booking` and `Space` objects or the internal `List<Proposal>`	A caller cannot clear or add stored proposals, or later reach objects that D2 makes mutable (occupancy, proposal status). This is acceptance case D1-F.
Domain objects are built from the supplied `Fixture` in the constructor	Hard-coding `new Booking("B1", …)` inside `StudentApplication`	The fixture stays the single source of truth, and its data passes through the same validating constructors as any other input.

4. Operation contracts

4.1 `StudentApplication.propose(String targetId): ProposalView`

- **Precondition:** none beyond the call. `targetId` may be null, malformed or unknown; each case is handled.
- **Postcondition on success:** `targetId` is a valid identifier, names a known space, and `EligibilityPolicy.isEligible(booking, target)` is true. A new `Proposal` is appended to the stored proposals with id `"P" + n` (n is the counter's previous value), booking id `B1`, target `targetId`, observed booking version equal to the booking's current version (0), observed policy version `policy.getVersion()` (1), and status `PENDING`. The method returns that proposal's `ProposalView`.
- **Postcondition on failure:** throws `IllegalArgumentException` when `targetId` is malformed, unknown or ineligible.
- **Frame (what may change):** only the proposal list (one append) and the id counter (+1), and only on success. The booking (its space, version and every other field), every `Space` and every existing proposal are unchanged in all cases.

4.2 `EligibilityPolicy.isEligible(Booking booking, Space target): boolean`

- **Precondition:** `booking` and `target` are non-null; otherwise `IllegalArgumentException`.
- **Postcondition:** returns true exactly when `!target.getId().equals(booking.getSpaceId())`, `target.isOpen()` and `!target.isOccupied()`. `booking.requiresAccessible()` does not affect the result under version 1. For the fixture, B12 and C03 are eligible; A17 and D09 are not.

- **Frame:** changes nothing; it is a query.

5. Most important current limitation

A proposal stops at `PENDING` and nothing re-checks it. There is no operator approval, rejection or execution yet, so the observed booking and policy versions stored in each proposal are never compared with the current state, and a proposal that has gone stale cannot be detected. Every rejection is also the same `IllegalArgumentException`, distinguished only by its message. D2 must add the operator actions, revalidate versions and eligibility inside one sequential boundary before committing, and return distinct outcomes (`NOT_APPROVED`, `STALE`, `UNAVAILABLE_OR_INELIGIBLE`, `COMMITTED`, `DUPLICATE`).

Other limitations: one in-memory fixture with no persistence; a single sequential process, so no safety under concurrent access; and accessibility is deliberately ignored until policy version 2 in D3.

6. Reused infrastructure and implemented parts

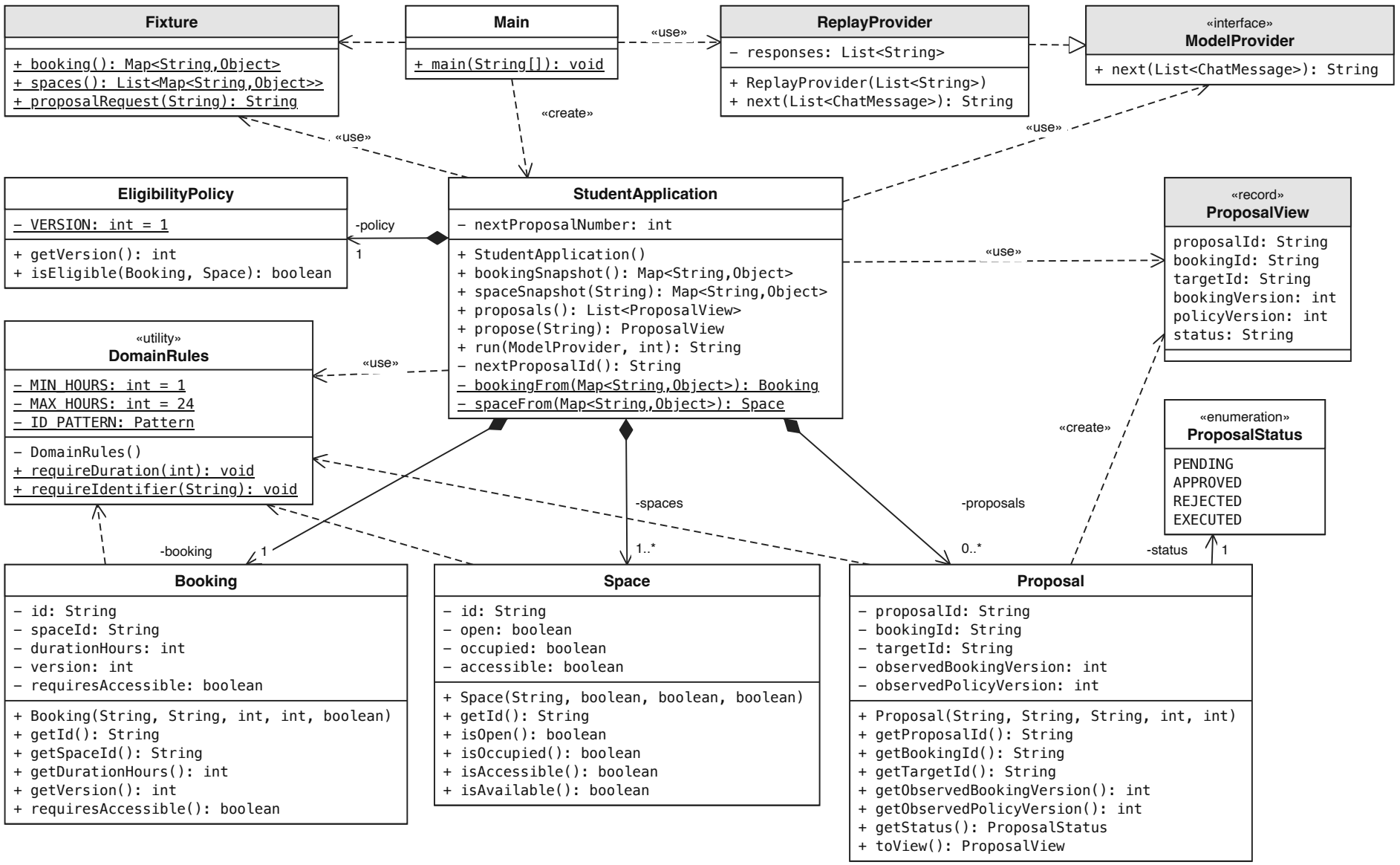
Reused unchanged from the 2026-09-15 starter: `ChatMessage`, `Fixture`, `MiniJson`, `ModelProvider`, `OllamaProvider`, `ProposalView`, `ReplayProvider`, the build scripts (`run.sh`, `run.ps1`, `setup-junit.sh`, `setup-junit.ps1`), `data/` and the supplied checks (`PublicChecks`, `InfrastructureChecks`, `SuppliedChecksTest`, `InfrastructureTest`).

Implemented for D1: `Booking`, `Space`, `Proposal`, `ProposalStatus` and `EligibilityPolicy`; the two validators in `DomainRules`; the domain storage, snapshots and `propose` in `StudentApplication` (its `run` method remains the D2 placeholder); the three-target `scripted` demonstration in `Main`; and 40 JUnit tests in `StudentTests`.

7. Testing

`bash run.sh check` runs the 14 supplied public checks as plain Java, and `bash run.sh test` runs 55 JUnit Jupiter tests: the same 14 checks, the infrastructure test and 40 student tests. All pass (`results/d1.txt`, 4 October 2026). Every rejection test asserts the unchanged booking, occupancy and proposal list after the exception, not only that an exception was thrown. `TEST_PLAN.md` maps acceptance cases D1-A to D1-F to their tests.

Figure 1. Class diagram: the D1 domain and application classes and the starter classes they use.

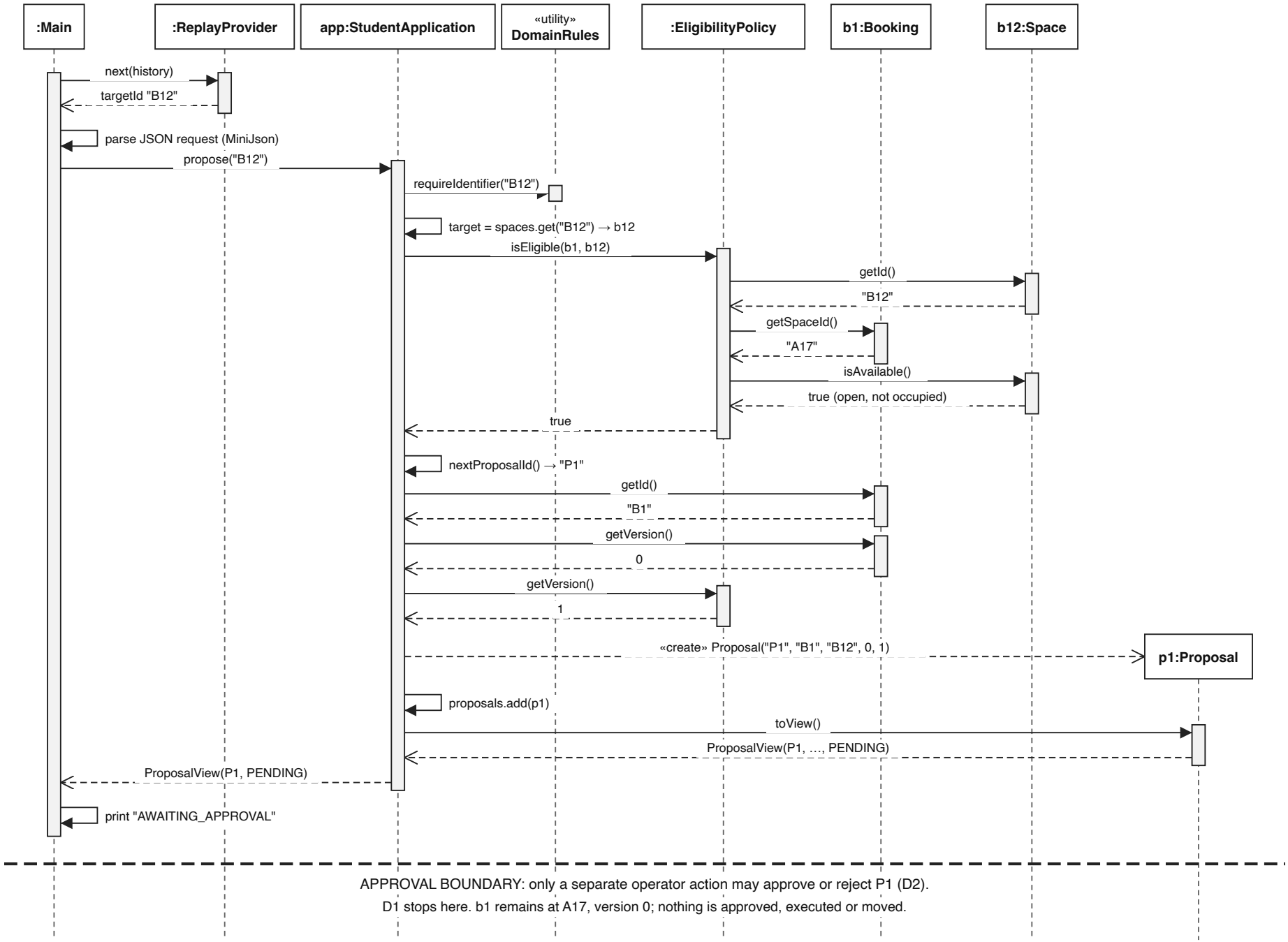


Grey header: supplied starter class, unchanged. White header: written or completed for D1.

Underlined members are static. Filled diamond: composition (StudentApplication owns the part). Solid line, open arrow: navigable association.

Dashed line, open arrow: dependency. Dashed line, hollow triangle: realization of an interface. Booking, Space, Proposal and StudentApplication validate through DomainRules.

Figure 2. Sequence diagram: reading B1, evaluating B12 and creating pending proposal P1 (`bash run.sh run scripted`, first request), with the approval boundary.



Solid line, filled arrow: call. Dashed line, open arrow: return or «create». Shaded bars: activations. Only calls with a visible effect on this path are shown.